

Writing a Classic Blogger Theme: A Code-Focused Tutorial

Blogger themes are defined in a single XML template that combines standard HTML markup with special Blogger tags ¹. This one file describes the layout and style for all page types (main index, post pages, archive pages, etc.). In practice, you write an HTML document with a few required attributes, then intermix Blogger-specific tags (prefixed with `<b:>`) and data placeholders (prefixed with `<data:>`) alongside normal HTML.

Structure: Template, Head, Body, and Blogger Tags

Begin with the XML prolog and doctype, then an `<html>` element including Blogger-specific attributes and XML namespaces. For example, you should set `b:layoutsVersion='3'` on `<html>` to enable the latest features like `<b:template-skin>` ². You can also disable Blogger's default CSS/JS by adding `b:css='false'` and `b:js='false'` to remove the built-in stylesheet and scripts ³ (this gives you full control over styling). Ensure to include the Blogger namespaces for `b`, `data`, and `expr` on the `<html>` tag – these enable the Blogger tags, data variables, and dynamic expressions in attributes ⁴.

In the `<head>`, use standard HTML (e.g. `<title>`) and Blogger includes for critical metadata. For instance, Blogger provides an includable named `"all-head-content"` that injects necessary meta tags, feed links, etc., which you include via `<b:include data='blog' name='all-head-content' />` ⁵. You'll also add a `<b:skin>` section here (discussed later) containing your CSS and theme variables. The `<body>` of the template contains your page structure, which you define using normal HTML elements and Blogger layout tags. Blogger's template system lets you define layout sections with `<b:section>` and widgets with `<b:widget>`. **Important:** a `<b:section>` can only contain `<b:widget>` tags (no direct HTML inside) ⁶. Each section acts as a container in which Blogger (or the blog owner via the Layout editor) can place widgets. You can have multiple sections (e.g. a sidebar, footer, main content), each with an `id` and optional attributes like `class`, `showaddelement` (whether the user can add more gadgets in that section via Blogger UI), etc. By convention, a main section contains the primary blog posts widget (type `"Blog"`), while other sections might hold widgets like `"Header"`, `"PageList"` (for nav links), etc.

Below is a minimal template structure. It defines two sections: a sidebar (where new widgets can be added) and a main area with a locked Blog widget for posts:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE html>
<html b:layoutsVersion='3' b:css='false' b:js='false'
      xmlns='http://www.w3.org/1999/xhtml'
      xmlns:b='http://www.google.com/2005/gml/b'
      xmlns:data='http://www.google.com/2005/gml/data'
      xmlns:expr='http://www.google.com/2005/gml/expr'>
  <head>
```

```

<title><data:blog.title/></title>
<b:include data='blog' name='all-head-content' />
<b:skin><![CDATA[
    /* Theme CSS and variables will go here (see later sections) */
]]></b:skin>
</head>
<body>
    <!-- Sidebar section: user can add gadgets here via Blogger Layout UI -->
    <b:section id='sidebar' class='sidebar' showaddelement='yes'
growth='vertical'>
        <!-- (widgets will be added dynamically, so left empty) -->
    </b:section>
    <!-- Main content section: contains the primary Blog widget (posts) -->
    <b:section id='main' class='main' showaddelement='no' growth='vertical'>
        <b:widget type='Blog' id='Blog1' locked='yes' />
    </b:section>
</body>
</html>

```

Most of the template is just like normal HTML, and any Blogger-specific tags you add will be processed by the engine into appropriate HTML elements (often as `<div>`s or included content) ⁷. In the example above, the Blog widget is left self-contained (`<b:widget .../>` with no children) which means it will use Blogger's default post markup. You could also customize it by providing child `<b:includable>` templates inside the widget, but using the default is a good starting point. The key takeaway is that the Blogger theme structure defines **where** content and widgets go (via sections/widgets), while the actual content (posts, gadgets, etc.) will be rendered by those widgets at runtime.

Blogger Logic: Variables, Conditionals, and Includes

Blogger templates support a simple logic/templating language via the `<b:...>` tags. You can insert dynamic data with the `data:` syntax, use conditionals, and include reusable snippets. **Variables:** Many data variables are provided by the platform – for example, `data:blog.title` is the blog's title, `data:blog.pageTitle` is the current page's title, `data:blog.homepageUrl` the home URL, etc. Inside certain contexts (like within a post loop or a widget), context-specific data becomes available (e.g. `data:post.title` for a post's title). To output a variable, use a self-closing data tag, e.g. `<data:blog.title/>` outputs the title text ⁸. In attribute values, use the `expr:` prefix to evaluate an expression – for example: `<a expr:href='data:blog.homepageUrl'>Home</a>` will set the `href` to your blog's URL.

You can also define your own variables within the template. The `<b:with>` tag creates a new scoped variable for its content: for example, `<b:with value='data:blog.title' var='siteName'>...</b:with>` would let you use `<data:siteName/>` inside to refer to the blog title. This is useful to avoid repeating complex expressions or to shorten references.

Conditionals: Use `<b:if>` to conditionally display content. The `cond` attribute takes a boolean expression (using `data:` variables and operators like `==`, `!=`, `<`, `>` etc.). There is a self-closing `<b:else/>` tag for the "false" branch of the if ⁹. (Note: if you need an "else if", you either nest another `<b:if>` or use the newer `<b:elseif>` tag in sequence). For example, to show a note only on item (post) pages:

```

<b:if cond='data:blog.pageType == "item"'>
  <h3>This is a post page.</h3>
<b:else/>
  <h3>This is not a post page.</h3>
</b:if>

```

In the above, `data:blog.pageType` is checked – if it equals `"item"` (a post page), we render one block, otherwise the `<b:else/>` clause handles other page types. (We use single quotes around the cond value and double quotes for the string "item" inside it, which is the proper way to include a string in the expression.)

Includes: The `<b:include>` tag pulls in a named chunk of template (an *includable*) at that location. It has two attributes: `name` (the name of the includable to include) and an optional `data` (to pass a data context into that snippet) ¹⁰. Blogger itself uses includes for required parts of the template (as we saw with `all-head-content`). You can define your own reusable fragments using `<b:includable id="...">...</b:includable>` within widgets, and then include them by name. For instance, if you had `<b:includable id='FooterNote'>...</b:includable>` defined (say, in your main section or a widget), you could insert it anywhere via `<b:include name='FooterNote' />` and that content will be injected. When using `b:include` inside a loop or widget, you can pass a data item: e.g., `<b:include name='postSnippet' data='post' />` would include a snippet template with access to a `data:post` variable. (In fact, the default Blog widget uses exactly this approach: it defines an includable for each post and includes it inside a loop – more on loops below.)

Loops and Data Iteration

Dynamic lists (like blog posts, comments, labels, menu items, etc.) are handled with the `<b:loop>` tag. **Loops** allow you to iterate over an array of items and output a template for each item. The `b:loop` tag requires two attributes: `values` (the collection to loop over) and `var` (a name for the loop variable for each item) ¹¹. Many Blogger data objects are arrays – for example, on the main blog page the Blog widget provides `data:posts` (an array of post objects), a PageList widget provides `data:links` (array of link items), a Label widget might have `data:labels`, etc. ¹². By using `b:loop`, you can generate HTML for each element in such arrays.

Inside a loop, you refer to the current item's properties via the loop variable. For instance, if `var='post'`, you can use `<data:post.title/>`, `<data:post.url/>`, etc., inside the loop to get each post's title, URL, and so on. You can even nest loops (for example, loop through `data:post.labels` inside a post loop to list each label). Here's a simple example that iterates over posts and displays a linked title and snippet for each:

```

<b:loop values='data:posts' var='post'>
  <h2><a expr:href='data:post.url'><data:post.title/></a></h2>
  <p><data:post.snippet/></p>
</b:loop>

```

In this code, `data:posts` is the array of post objects (provided by the Blog widget on index pages). We loop through it with `var='post'`. Inside, we use `<data:post.title/>` to output the post title, and use an anchor tag with `expr:href='data:post.url'` to link to the post's URL. We also output a short snippet of the post via `<data:post.snippet/>` (which shows an excerpt if the post is truncated

by a jump break, or the beginning of the post by default). **Note:** The actual properties available on `data:post` are quite rich (e.g. you have `data:post.content`, `data:post.date`, `data:post.labels`, etc.). You can conditionally display elements inside the loop as needed (for example, only show a "Read more" link if `data:post.hasJumpLink` is true, using a `<b:if>` around the link).

Similarly, you can loop through other data sets. For example, in a `PageList` widget (which lists navigation links), you might see: `<b:loop values='data:links' var='item'> ... </b:loop>` to iterate through each link item ¹³. Inside such a loop, `data:item.title` and `data:item.href` could be used to build a list of links. Loops are fundamental to rendering lists of items in the theme.

Frontend Customizer: Theme Skins and Live Customization

One powerful feature of Blogger's classic themes is the **Theme Designer integration**, which lets users customize colors, fonts, etc., without editing code. To enable this, your template defines **theme variables** and CSS inside the `<b:skin>` tag. The `<b:skin>` section (which must be placed in the `<head>` of the template) contains all your CSS, wrapped in a CDATA section so that the XML parser ignores it ¹⁴. Within this CSS block, you declare customizable variables using the `<Variable>` tag (note: this tag is used inside the skin and does not carry a `b:` prefix). Each variable has attributes like `name` (a unique identifier), `type` (commonly `"color"` or `"font"`), `default` and `value` (the default and current value), and an optional `description` (for the UI) ¹⁵ ¹⁶. These variables become available in the Blogger Theme Designer (under **Advanced** customization) for the blog owner to tweak ¹⁷. In your CSS, you use the variable's value by writing `$(variableName)` which will be replaced with the chosen value ¹⁸ ¹⁹.

For example, let's define a custom background color variable and use it in the CSS. We'll also demonstrate a snippet of JavaScript to allow live tweaking of this color on the page (for preview purposes):

```
<!-- In HEAD: define a theme variable and use it in CSS -->
<b:skin><![CDATA[
  <Variable name="mainBgColor" description="Background Color" type="color"
  default="#ffffff" value="#ffffff"/>
  body { background-color: $(mainBgColor); }
]]></b:skin>

<!-- Later in BODY: an interactive control to live-preview changes -->
<input type="color" id="bgPicker" value="#ffffff"/>
<script>
  document.getElementById('bgPicker').addEventListener('input', function() {
    document.body.style.backgroundColor = this.value;
  });
</script>
```

In the `<b:skin>` above, we declared `mainBgColor` as a color variable (defaulting to white). In the CSS, `body { background-color: $(mainBgColor); }` means the body's background will use that variable's value. The Blogger engine will substitute `$(mainBgColor)` with the actual color code (for instance, if the user picks a new color in the Theme Designer, that value is stored and applied). The `<input type="color">` and script are **not** a built-in requirement – we added them just to

demonstrate live customization. This snippet creates a color picker on the page that, when adjusted, immediately updates the page's background color via DOM scripting. It's a simple way to preview changes in real time. In a real template, the user would normally change `mainBgColor` through Blogger's Theme Designer interface (or by editing the code), but adding such controls can be a nice touch for theme previews or dynamic themes.

Finally, a word on `<b:template-skin>`: Blogger allows a separate skin section specifically for the Layout Editor (the drag-and-drop layout preview in Blogger's dashboard). The `<b:template-skin>` tag, if used, should also be placed in the head (after `<b:skin>`) and contain CSS/variables that apply **only in the Blogger Layout view** ²⁰ ²¹. This is useful if you want certain styling in the editor (like placeholder images or outlines) that shouldn't appear on the live blog. In most cases, you can omit `<b:template-skin>` or keep it minimal, but it's good to know it's available. Remember that `<b:template-skin>` requires the `b:layoutsVersion='3'` attribute on the html tag (as we set earlier) to function ².

Summary: A classic Blogger theme is essentially an HTML5 layout enhanced with Blogger's own XML tags for logic and data. Use `<b:section>` and `<b:widget>` to structure the page, output dynamic content with `data:` variables, control flow with `<b:if>` (and `<b:else/>`), loop through lists with `<b:loop>`, and include reusable chunks with `<b:include>`. Define a `<b:skin>` with CSS and `<Variable>` definitions so users can customize colors/fonts easily ²². With these building blocks, you can code a fully custom Blogger template. The key is to leverage Blogger's provided data (posts, widgets, etc.) and blend it with your HTML/CSS. As an expert developer, you'll find the syntax straightforward – it's basically XML with some extra tags. Happy theming!

Sources: The above insights and examples are based on official Blogger documentation and community tutorials ⁴ ¹⁰ ¹¹ ⁹ ¹⁴, ensuring that the code and practices shown are correct and up-to-date.

¹ ⁴ ⁵ ⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹³ ¹⁴ ²² Structure of Custom Blogger Themes - GeekThis
<https://geekthis.net/post/structure-of-custom-blogger-themes/>

² html - blogger error bX-d55p58 - Stack Overflow
<https://stackoverflow.com/questions/46539206/blogger-btemplate-skin-error-bx-d55p58>

³ How to remove Blogger CSS widget bundle - Stack Overflow
<https://stackoverflow.com/questions/16911018/how-to-remove-blogger-css-widget-bundle>

¹⁵ ¹⁶ ¹⁷ Blogger Template Design — CSS Styling and Layout using Variables — Part 4 | by aqeelakhtar.sdc | Medium
<https://medium.com/@aqeelakhtar.sdc/blogger-template-design-css-styling-and-layout-using-variables-part-4-d8d8681f504e>

¹⁸ ¹⁹ My Blogger Template · GitHub
<https://gist.github.com/revathskumar/3454166>

²⁰ ²¹ Blogger b:skin and b:template-skin Tag | BloggerBasics101
<https://bloggerbasics101.blogspot.com/2018/11/blogger-bskin-and-btemplate-skin-tag.html>